Enterprise Analytics Hackathon

AdventureWorks Analytics Layer — Project Documentation

Author: Umer Sahi
Week 3 — Friday Hackathon
Source system: AdventureWorks (OLTP) on PostgreSQL

This document consolidates the database overview, setup instructions, analytics architecture, full data dictionary, SQL design decisions, challenges resolved, assumptions made, notebook contents, and executive recommendations for the project.

1. Database Overview

The source database is **AdventureWorks (OLTP)**, restored from the provided CSV extract using the community **AdventureWorks-for-Postgres** install script into PostgreSQL. It is a transactional (not analytical) schema spread across five namespaces:

Schema	Domain
sales	Orders, order lines, customers, salespeople, territories, stores
production	Products, categories/subcategories, inventory, locations
purchasing	Purchase orders, purchase order lines, vendors
humanresources	Employees
person	People (individuals behind customers/employees/vendors)

Loaded volume: 31,465 sales orders / 121,317 order lines / 19,820 customers / 504 products / 4,012 purchase orders / 8,845 purchase order lines / 104 vendors, spanning **2022-05-30 to 2025-06-29** (~3 years).

This is a standard, well-documented sample database, so the pipeline uses its real table/column names throughout rather than a generic mock schema.

2. Setup — How to Reproduce This Project

2.1 Restore the raw AdventureWorks database

- Download the AdventureWorks CSV data extract and the community **install.sql** port for Postgres.
- Run the Ruby conversion script (**update_csvs.rb**) once, to reformat the CSVs for Postgres compatibility.
- Create an empty database, then run the install script via **psql** (not pgAdmin's Query Tool, since it relies on psql-only meta-commands like `\copy`):

```
psql -U postgres -d Adventureworks -f install.sql
```

2.2 Build the analytics layer

With the raw schema populated, run the staged pipeline once, top to bottom. It is fully idempotent (every `CREATE TABLE` is preceded by `DROP TABLE IF EXISTS`), so it is safe to re-run after any fix.

```
psql -U postgres -d Adventureworks -f analytics_pipeline.sql
```

2.3 Run the notebook

- Install dependencies: `pip install notebook pandas sqlalchemy psycopg2-binary matplotlib seaborn`
- Launch Jupyter Notebook locally and open **executive_analysis.ipynb**.
- Update the connection credentials (DB_USER / DB_PASSWORD / DB_HOST / DB_PORT / DB_NAME) at the top of the notebook to match the local environment.

- Run all cells top to bottom — every query reads exclusively from the `analytics.*` schema, never the raw operational tables.

3. Analytics Architecture

Everything reusable lives in a new **analytics** schema, built as a strict dependency chain — each stage is a `CREATE TABLE ... AS SELECT` that only reads from tables built in an earlier stage. The raw operational schemas are queried exactly once (Stage 1 & 2) and never again downstream.

Stage	Name	Tables / Views Produced
1	Dimensions	dim_date, dim_product, dim_customer, dim_territory, dim_employee_salesperson
2	Fact Tables	fact_sales_line, fact_purchase_line, inventory_snapshot_analytics
3	Domain Analytics	customer_analytics, product_analytics, employee_sales_analytics, territory_sales_analytics, vendor_purchasing_analytics
4	Business Metrics	monthly_revenue, quarterly_revenue, product_performance_ranking, category_performance
5	Segmentation	customer_segments, customer_retention_summary
6	Regional Analysis	regional_performance
7	People & Ops	salesperson_rankings, inventory_health, purchasing_trends
8	Executive KPIs	executive_monthly_kpi, executive_kpi_summary

25 analytical tables/views across **7 business domains** (customer, product, sales, employee, territory, inventory, purchasing/vendor) — well above the 10-table / 5-domain requirement. `fact_sales_line` is the single source of truth for revenue/cost/margin: every downstream table derives its numbers from it, so the same figure is never recalculated with slightly different logic twice. The same principle applies to `fact_purchase_line` for purchasing/vendor data.

4. Data Dictionary

Every table below is built with `CREATE TABLE ... AS SELECT` in **analytics_pipeline.sql**, in the stage order shown. Row counts are a verification snapshot from the last full pipeline run.

Table	Stage	Rows	Purpose
dim_date	1	1,857	Generated date spine (day grain) covering the order-date range padded by a year each side.
dim_product	1	504	Product master enriched with category/subcategory rollup and computed list-price margin %.
dim_territory	1	10	Sales territory lookup (name, country/region, sales group).
dim_customer	1	19,820	Customer resolved to one display name whether they are an individual or a store account.
dim_employee_salesperson	1	17	Salesperson joined to person + territory, with quota and commission rate.
fact_sales_line	2	121,317	Order-line grain fact; revenue/cost/margin recomputed from qty × unit price × (1 – discount).
fact_purchase_line	2	8,845	Purchase-order-line grain fact with lead time and rejection quantities.
inventory_snapshot_analytics	2	1,069	Current on-hand inventory per product/location.
customer_analytics	3	19,119	One row per purchasing customer: orders, revenue, AOV, recency.
product_analytics	3	504	One row per product: units sold, revenue, cost, realized margin %.
employee_sales_analytics	3	17	One row per salesperson: revenue, customers served, quota attainment (latest full year).
territory_sales_analytics	3	10	One row per territory: orders, customers, salespeople, revenue, margin.
vendor_purchasing_analytics	3	86	One row per vendor: spend, lead time, rejection rate, preferred status.
monthly_revenue	4	38	Monthly revenue/margin with LAG()-based MoM growth and 3-month moving average.
quarterly_revenue	4	13	Quarterly rollup with QoQ/YoY growth.
product_performance_ranking	4	504	RANK()/NTILE(4) product rankings overall and within category, with Top/Bottom 10 tags.
category_performance	4	4	Category-level revenue share via conditional/window aggregation.
customer_segments	5	19,119	RFM scoring via NTILE(5) and CASE WHEN segment classifier (Champions/Loyal/At Risk/etc.).
customer_retention_summary	5	6	Segment-level rollup of customer count and revenue share.
regional_performance	6	10	Territory revenue/margin with RANK() and full-year-only YoY growth.
salesperson_rankings	7	17	RANK()/DENSE_RANK() salesperson rankings vs. team-average revenue.
inventory_health	7	432	CASE WHEN stock status per product vs. its own safety stock/reorder point.
purchasing_trends	7	31	Monthly purchasing spend and lead time with LAG()-based MoM growth.
executive_monthly_kpi	8	38	Wide monthly trend table joining revenue and purchasing metrics.
executive_kpi_summary	8	1	Single-row headline scorecard pulling from every prior stage.

Note: customer_analytics / customer_segments (19,119) is smaller than dim_customer (19,820) by design — only customers with at least one order are included, since the ~700 zero-order customer records have nothing to analyze.

5. SQL Design Decisions

- **Recomputed, not trusted, revenue.** fact_sales_line.line_revenue is calculated from OrderQty × UnitPrice × (1 – UnitPriceDiscount) rather than copied from the raw LineTotal column, giving one auditable formula for every downstream table. It was reconciled against SalesOrderHeader.SubTotal (off by ~$0.50 across $109.8M — pure rounding) before anything was built on top of it.
- **Tables, not views.** Given the chained, multi-stage design, materialized CREATE TABLE AS SELECT was used instead of plain views so each stage is fast to query and easy to inspect independently. The whole file is idempotent (DROP TABLE IF EXISTS before every CREATE TABLE), so it can simply be re-run.
- **Indexes** were added on every foreign-key-style join column (order_date, customerid, productid, territoryid, salespersonid, etc.) since these tables are meant to be queried repeatedly by a dashboard, not just read once.
- **Sanity checkpoints** are included after every stage (SELECT COUNT(*) summaries) so a re-run of the file surfaces a broken join immediately instead of silently producing an empty or malformed table.

Advanced SQL concepts demonstrated

- **Window functions** — RANK(), DENSE_RANK(), NTILE(4)/NTILE(5), LAG(), moving averages (39 uses across the file).
- **Chained CTEs** — 21 WITH clauses, each stage building on the previous instead of re-deriving metrics.
- **CASE WHEN & conditional aggregation** — stock status, segment classification, tier tagging, quota logic.
- **Complex joins** — multi-table joins resolving customer identity, product category rollups, and territory attribution.

6. Challenges Faced (and how they were resolved)

6.1 Partial boundary years distorting territory YoY growth

A first pass at regional_performance compared calendar year 2025 (data through June only) against full-year 2024, producing nonsensical “growth” figures like -70%. **Fix:** regional_performance now only compares years where all 12 months are present in the data.

6.2 Quota attainment compared against the wrong time window

An early version of employee_sales_analytics divided each salesperson's 3-year cumulative revenue by their annual SalesQuota, producing absurd “4,000% attainment” figures. **Fix:** quota_attainment_pct now compares quota against revenue in the latest complete calendar year only. Even after that fix, quotas ($250K–$300K) turned out to be far below realistic revenue per salesperson (~$2M–$4M/year) for this dataset — every salesperson with a quota clears it — so this is treated as a genuine data-limitation finding (stale quotas) rather than a performance story, using team-average revenue comparison as the more meaningful ranking instead.

6.3 Partial final month in the data

The final month in the data (June 2025) is a partial month (905 orders vs. ~2,200 typical for surrounding months) because the extract simply stops mid-month. executive_kpi_summary reports this month separately (partial_month_in_data) rather than folding it into the headline “latest month” growth figure.

6.4 Duplicate rows in the raw product table

A re-run of the CSV import (without dropping the database first) left duplicate productid rows in production.product, which broke the dim_product primary key creation. **Fix:** dim_product now builds with SELECT DISTINCT ON (productid) ... ORDER BY productid, and the underlying schemas were re-imported cleanly to eliminate the root cause.

7. Assumptions Made

- **“Customer” analytics covers purchasing customers only** — customers with zero historical orders are excluded from customer_analytics / customer_segments, since they'd have undefined recency/frequency/monetary values otherwise.
- **Customer Lifetime Value** is a simple historical proxy (total revenue to date), not a predictive/discounted model — a true CLV model would need churn probability and time-value-of-money assumptions this dataset doesn't support out of the box.
- **RFM segment boundaries** (Champions / Loyal / At Risk / etc.) use a standard, commonly-used quintile-based scoring scheme; a real business would likely tune the thresholds against their own historical win-back/churn outcomes.
- **“Full year” for YoY comparisons** is defined as any calendar year with all 12 months represented in fact_sales_line/monthly_revenue — this automatically excludes 2022 (data starts in May) and 2025 (data ends in June) without hardcoding specific years.
- **Inventory health thresholds** (Out of Stock / Low Stock / Adequate / Healthy / Overstocked) are based on each product's own SafetyStockLevel and ReorderPoint fields already present in production.Product, rather than an external inventory policy.

8. Python Analytics Notebook

executive_analysis.ipynb is the presentation layer of the pipeline. It connects to PostgreSQL via SQLAlchemy/psycopg2 and reads exclusively from analytics.* via pandas.read_sql — it never queries the raw operational tables directly. It produces **10 visualizations** (exceeds the 8-chart requirement), each followed by a data-driven business insight:

- Revenue Trend (Monthly) — from monthly_revenue, with 3-month moving average.
- Sales by Territory — from regional_performance.
- Customer Segments (RFM) — from customer_segments / customer_retention_summary.
- Product Performance Ranking — from product_performance_ranking.
- Category Revenue Share — from category_performance.
- Employee / Salesperson Performance — from salesperson_rankings.
- Inventory Health Status — from inventory_health.
- Purchasing Trends — from purchasing_trends.
- Quarterly Revenue & Growth — from quarterly_revenue.
- Executive KPI Scorecard — from executive_kpi_summary.

9. Executive Recommendations

■ Business Opportunities

- Grow the international territories — Germany, France, and the UK posted the strongest full-year revenue growth (367%, 145%, 131% respectively) off a small base.
- Win back the “At Risk (was loyal)” segment — 2,368 customers still represent $21.19M (19.3% of revenue) in historical value.
- Protect and grow the Champions segment — 12.7% of customers drive 69.2% of revenue.
- Push Bikes — already 86.2% of revenue; bundling underperforming categories with it could lift attach rate.
- Right-size overstocked inventory — $183,001 tied up in 50 overstocked SKUs.

■ Business Risks

- Negative-margin territories — Central, Southeast, and Northeast run negative realized margin despite real revenue.
- Revenue concentration in Champions — losing even a handful of these accounts would materially hurt revenue.
- Stockout risk — 8 products are out of stock or at/below reorder point.
- Uneven rep performance — 10 of 17 salespeople sit below the team-average revenue (note: quotas themselves are stale and not a usable benchmark).
- Declining average order value — revenue per order fell markedly from mid-2024 onward even as order count held up.

■ Actionable Recommendations

- Audit pricing/discounting in Central, Southeast, and Northeast to find the source of negative margins.
- Launch a win-back campaign for the “At Risk” segment, prioritized by historical spend, using the RFM scores in customer_segments.
- Resupply the 8 at-risk SKUs immediately and review reorder points quarterly using inventory_health.
- Increase sales investment in Germany, France, and the UK while investigating the lowest-margin US territories.
- Reset sales quotas to current revenue scale and refresh this pipeline monthly so executive_kpi_summary stays a trustworthy, live source of truth.

10. Deliverables & Future Work

- **analytics_pipeline.sql** — the full, staged, commented SQL pipeline. Idempotent; safe to re-run top to bottom.
- **executive_analysis.ipynb** — connects to PostgreSQL, reads exclusively from analytics.*, produces 10 visualizations with business insights and executive recommendations generated from live query results.
- **README.md** — architecture and design-decision reference.

- **This document** — consolidated project documentation (PDF).

Future Work (Bonus Challenge)

Because every table in analytics.* only ever reads from earlier analytics.* tables, a new dashboard can be built entirely on top of this layer without touching the raw schemas. Two natural next steps: (1) convert the CREATE TABLE AS statements to a scheduled job (cron + psql -f analytics_pipeline.sql, or Postgres materialized views refreshed with REFRESH MATERIALIZED VIEW) so the layer updates automatically; (2) add a lightweight analytics.pipeline_run_log table recording each run's timestamp and row counts, turning the existing sanity checkpoints into a permanent data-quality audit trail.